CodeBreaker

Ben Brackenbury
beb52@aber.ac.uk
May 2020

Table of contents

1 Introduction 3

1.1 Functional Requirements 3

1.2 UML Use Case Diagram 4

2 Design 5

2.1 UML Class Diagram 5

2.2 Classes & Relationships 6

2.3 Algorithm Example 8

3 Testing 8

3.1 Test Table 8

3.2 Command Line 21

4 Evaluation 22

4.1 Solving the assignment 22

4.2 Flair 22

4.3 Challenges 22

4.4 What remains to be done 22

4.5 What was learnt 22

4.6 Estimated mark 23

1 Introduction

In this report, I have documented the design, testing and evaluation processes of my assignment, with UML diagrams, pseudocode, and test tables.

Below, I have outlined the functional requirements of the application, presented as a table and a UML use case diagram. Section 2 involves the design of the application in terms of the classes uses and their relationships. This is presented with a UML class diagram and a discussion about each class and its relationship to others. Section 3 shows my testing plan, with results shown as screenshots, with a brief discussion on the input data used for the tests. Finally, Section 4 is a self-evaluation of the assignment, including how I found the task, my development process, and areas to improve upon.

I feel that I have completed the assignment, following all of the stepson the brief.

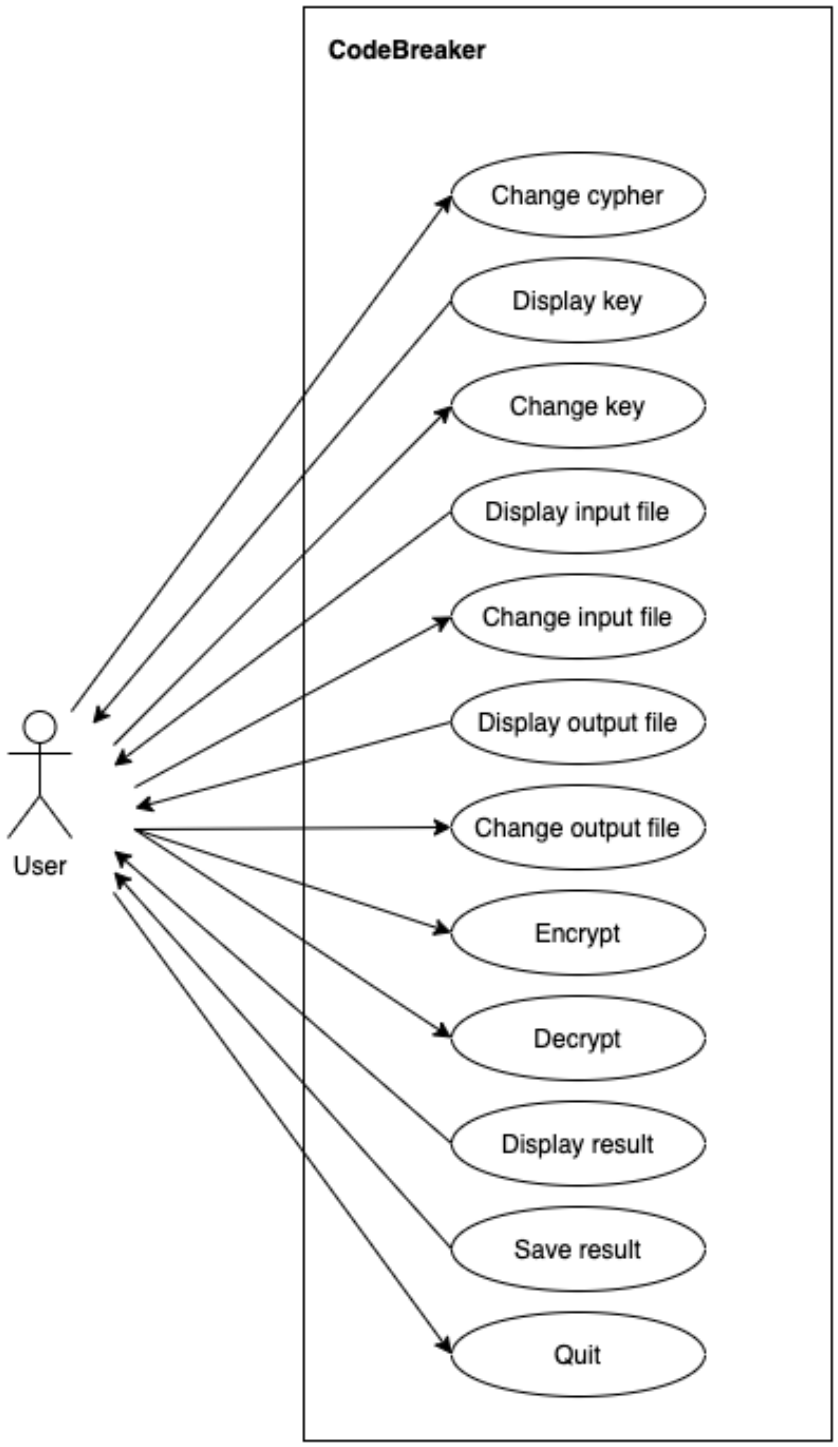
1.1 Functional Requirements

Table 1

ID	Description
FR1	Change cypher
FR2	Load key from file
FR3	Change key
FR4	Load input text from file
FR5	Change input file
FR6	Load output file
FR7	Change output file
FR8	Encrypt plaintext from input file
FR9	Decrypt cyphertext from input file
FR10	Save encryption/decryption result to output file
FR11	Quit application

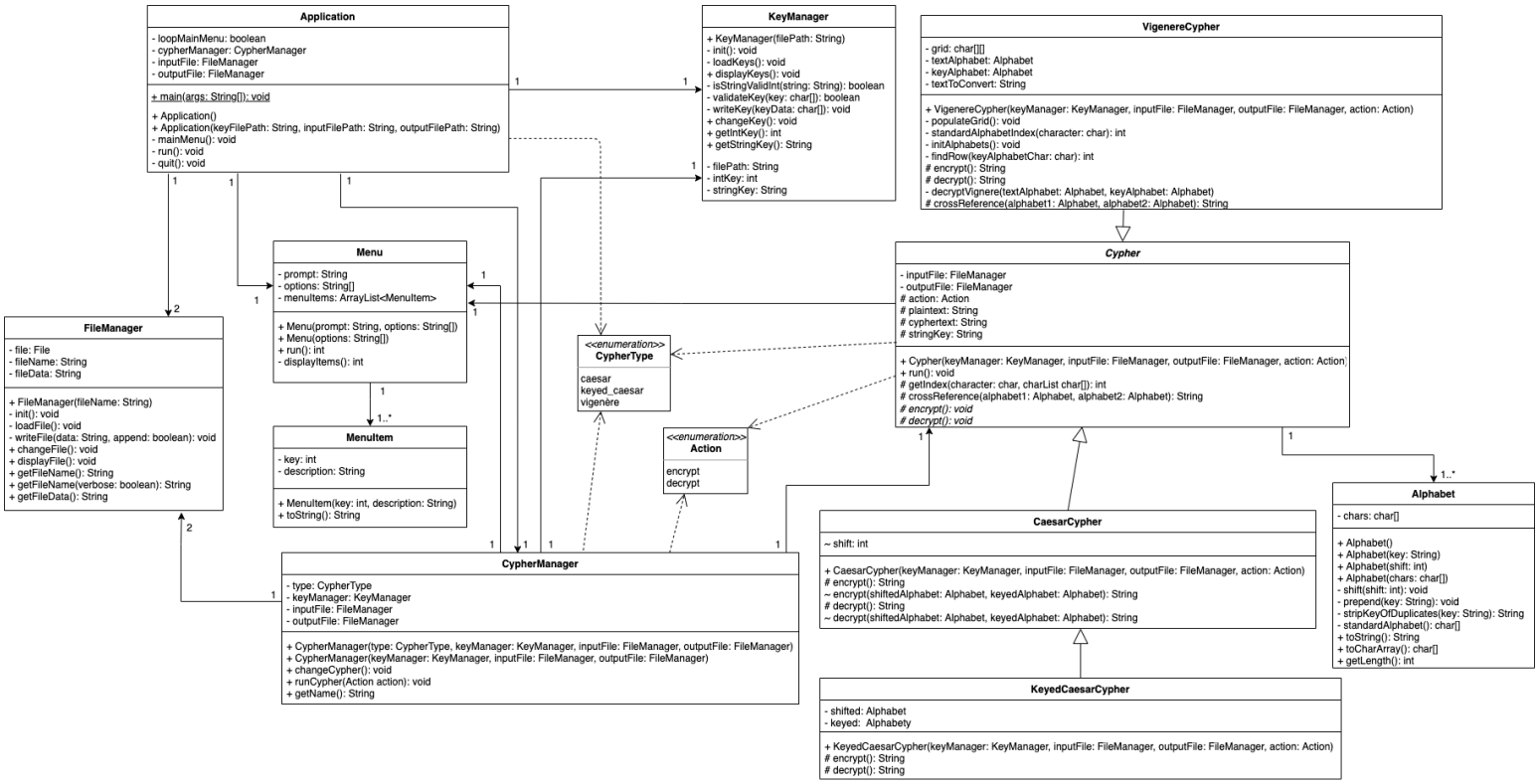
1.2 UML Use Case Diagram

Diagram 1



2 Design

2.1 UML Class Diagram



2.2 Classes & Relationships

Table 2

Package	Class	Description	Relationships
uk.ac.aber.beb52.codebreaker	**Application**	Main class that contains the application's *main* method. The *main* method created a new instance of the *Application* class, calling different constructors based on whether three file path arguments have been provided, and then call the *run method*. The *Application* constructors creates a *KeyManager* object and two *FileManager* objects for the key, input, and output files respectively. The method *run* displays the main menu until the application is exited.	The *Application* class creates a *KeyManager*, *CypherManager*, and *Menu* objects, as well as two *FileManager* objects. Public methods of these objects are called via options in the main menu.
uk.ac.aber.beb52.codebreaker.cyphers	**Alphabet**	The *Alphabet* class represents an alphabet of characters, such as a standard alphabet of the letters A-Z, a shifted alphabet, or an alphabet with a key prepended. The class contains methods for shifting alphabets, prepending keys, and printing alphabets as either strings or arrays of characters.	The *Alphabet* class is fundamental to the encryption and decryption of all cyphers, and is therefore implemented by *Cypher* and all of its subclasses.
	Cypher	*Cypher* is the superclass from which all cyphers inherit; it contains properties and methods that are common between all subclasses. The class's *encrypt* and *decrypt* methods are abstract and must be overridden by each subclass.	All cyphers are created by a *CypherManager* object. *Cypher* uses the Alphabet objects in its *CrossReference* method.
	VigenereCypher	The *VigenereCypher* class overrides the *encrypt* and *decrypt* methods from the superclass, using a grid (2d array) of characters.	*VigenereCypher* inherits from the *Cypher* class and uses Alphabet objects for encryption and decryption
uk.ac.aber.beb52.codebreaker.cyphers.caesar	**CaesarCypher**	Overrides the *encrypt* and *decrypt* methods from the *Cypher* superclass, comparing a standard A-Z alphabet with a shifted one.	*CaesarCypher* inherits from the *Cypher* class and uses Alphabet objects for encryption and decryption
	KeyedCaesarCypher	Inherits from the *CaesarCypher* class, comparing a shifted alphabet with one with a prepended key.	Keyed*CaesarCypher* inherits from the *CaesarCypher* class and therefor the *Cypher* class and uses Alphabet objects for encryption and decryption
uk.ac.aber.beb52.codebreaker.enums	**Action**	Enumeration for specifying either *encrypt* or *decrypt*	The *Action* enumeration is used by both *Cypher* and *CypherManager* to determine whether to encrypt or decrypt.
uk.ac.aber.beb52.codebreaker.managers	**CypherType**	Enumeration for specifying which cypher to use (*caesar* | *keyed_caesar* | *vigenère*)	The *CypherType* enumeration is used by *Application*, *Cypher* and *CypherManager* to determine which cypher to use.

Package	Class	Description	Relationships
uk.ac.aber.beb52.codebreaker.managers	**CypherManager**	Allows the user to change which cypher to use and creates the specified cypher, passing in the key, input and output files.	A *CypherManager* object is created by the *Application* class, and uses *KeyManager*, *Cypher*, *Menu*, and *FileManager* objects.
	FileManager	Handles the loading, saving, and modifying of files. Used for input and output files.	Two *FileManager objects are created by the Application* class and are used by a *CypherManager* object to handle input and output files.
	KeyManager	Handles the loading, saving, and modifying for the key file. Parses the file to separate they integer from the string section of the key.	A *KeyManager* object is created from the *Application* class. It is used by a *CypherManager* object to pass along key data to a *Cypher* object.
uk.ac.aber.beb52.codebreaker.menu	**Menu**	Handles the displaying and input of menus. Displays a list of menu items and prompts the user for an input.	*Menu* is used by *Application*, *Cypher* and *CypherManager* objects. It uses *MenuItem* objects to displaying handle options .
	MenuItem	Creates a menu item to use in a *Menu*. Consists of an integer key and a textual description.	*MenuItem* objects are used by *Menu* objects to dimply and handle options.

2.3 Algorithm Example

Vigenère Cypher encryption algorithm

```
for each char in plaintext:
     output = []
     position = index of char in plaintext
     keyAlphabetChar = character at index position of key alphabet
     column = index of char in A-Z alphabet
     for index i in 0 → 25:
          if grid*[i][0] == keyAlphabetChar:
               row = i
     append to output character at grid[row][column]
return output as string
```

* where grid is an array of shifted character arrays

I chose this algorithm because it involves a more steps than others and involves two loops (one nested inside the other) whereas encryption and decryptions algorithms for other cyphers only involve one.

3 Testing

3.1 Test Table

Table 3

ID	FR (See Table 1)	Description	Inputs	Expected output	Actual output (See Section 3.1.1)	Pass/ Fail
T1.1	FR1	Checking that the user is able to change which cypher to use, via the main menu	1 (change cypher), 2 (Keyed Caesar)	Main menu should display again with the text *Cypher: Keyed Caesar* displayed above	S1.1	Pass
T1.2			1 (change cypher), 3 (Vigenère)	Main menu should display again with the text *Cypher: Vigenère* displayed above	S1.2	Pass
T1.3			1 (change cypher), 1 (Caesar)	Main menu should display again with the text *Cypher: Caesar* displayed above	S1.3	Pass
T1.4			1 (change cypher), *a* (invalid input)	An error ”try again” will be displayed and the menu will be shown again	S1.4	Pass
T2.1	FR2	Checking that the key has been successfully loaded from the key file	Program run with no arguments, 2 (Display key)	As a key file has not been specified by the user, the program will default to loading the *keys* file in the project’s root directory. The program will display the integer key and string key parsed from the file	S2.1	Pass
T2.2	FR3	Checking that the user is able to change the key and that the file is updated	3 (change key), *4apple* (valid key)	The program should display “*Successfully changed key*”. When the Test 2.1 is run again, the program will display information from the new key	S2.2	Pass
T2.3			3 (change key), *mykey* (invalid key)	The program should display an error saying “*The provided key does not have a valid format, please try again*” and allow the user to try again	S2.3	Pass

ID	FR (See Table 1)	Description	Inputs	Expected output	Actual output (See Section 3.1.1)	Pass/ Fail
T3.1	FR4	Checking that the input file has been successfully loaded	Program run with no arguments, 4 (Display input file)	As an input file has not been specified by the user, the program will default to loading the *input* file in the project's root directory. The program will display the name and path of the file, along with its formatted contents (capitalised with punctuation removed)	S3.1	Pass
T3.2	FR5	Checking that the user is able to change the input file	5 (Change input file), *input2* (valid file path)	When the Test 3.1 is run again, the program will display information from the new input file	S3.2	Pass
T3.3			5 (Change input file), *myfile* (valid file path)	The program should display an error saying "*Unable to load file "myfile"*" and allow the user to try again	S3.3	Pass
T4.1	FR6	Checking that the output file has been successfully loaded	Program run with no arguments, 6 (Display output file)	As an output file has not been specified by the user, the program will default to loading the *output* file in the project's root directory. The program will display the name and path of the file, as well as its contents	S4.1	Pass
T4.2	FR7	Checking that the user is able to change the output file	7 (Change output file), *output2* (valid file path)	When the Test 4.1 is run again, the program will display information from the new output file	S4.2	Pass
T4.3			7 (Change output file), *myfile* (invalid file path)	The program should display an error saying "*Unable to load file "myfile"*" and allow the user to try again	S4.3	Pass
T5.1.1	FR8 FR10	Check that the program successfully encrypts the text from input file	Caesar cypher, 8 (Encrypt), 1 (Overwrite file)	The text from the input file will be encrypted with a shift of 7 and the result will be written to *output, overwriting the existing contents*	S5.1.1	Pass

ID	FR (See Table 1)	Description	Inputs	Expected output	Actual output (See Section 3.1.1)	Pass/ Fail
T5.1.2	FR8 FR10	Check that the program successfully encrypts the text from input file	Caesar cypher, 8 (Encrypt), 2 (Append to file)	The text from the input file will be encrypted with a shift of 7 and the result will be written to *output, appending to the existing contents*	S5.1.2	Pass
T5.2.1			Keyed Caesar cypher, 8 (Encrypt), 1 (Overwrite file)	The text from the input file will be encrypted with a shift of 7 and a key of *LEMON.* *T*he result will be written to *output, overwriting the existing contents*	S5.2.1	Pass
T5.2.2			Keyed Caesar cypher, 8 (Encrypt), 2 (Append to file)	The text from the input file will be encrypted with a shift of 7 and a key of *LEMON.* *T*he result will be appended to the *output* file	S5.2.2	Pass
T5.3.1			Vigenère cypher, 8 (Encrypt), 1 (Overwrite file)	The text from the input file will be encrypted with a key of *LEMON.* *T*he result will be written to *output, overwriting the existing contents*	S5.3.1	Pass
T5.3.2			Vigenère cypher, 8 (Encrypt), 2 (Append to file)	The text from the input file will be encrypted with a key of *LEMON.* *T*he result will be appended to the *output* file	S5.3.2	Pass
T6.1.1	FR9 FR10	Check that the program successfully decrypts the text from input file	Caesar cypher, 9 (Decrypt), 1 (Overwrite file)	The text from the input file will be decrypted with a shift of 7 and the result will be written to *output, overwriting the existing contents*	S6.1.1	Pass
T5.1.2			Caesar cypher, 9 (Decrypt), 2 (Append to file)	The text from the input file will be decrypted with a shift of 7 and the result will be written to *output, appending to the existing contents*	S6.1.2	Pass

ID	FR (See Table 1)	Description	Inputs	Expected output	Actual output (See Section 3.1.1)	Pass/ Fail
T6.2.1	FR9 FR10	Check that the program successfully decrypts the text from input file	Keyed Caesar cypher, 9 (Decrypt), 1 (Overwrite file)	The text from the input file will be decrypted with a shift of 7 and a key of *LEMON.* *T*he result will be written to *output, overwriting the existing contents*	S6.2.1	Pass
T6.2.2			Keyed Caesar cypher, 9 (Decypt), 2 (Append to file)	The text from the input file will be decrypted with a shift of 7 and a key of *LEMON.* *T*he result will be appended to the *output* file	S6.2.2	Pass
T6.3.1			Vigenère cypher, 9 (Decrypt), 1 (Overwrite file)	The text from the input file will be decrypted with a key of *LEMON.* *T*he result will be written to *output, overwriting the existing contents*	S6.3.1	Pass
T6.3.2			Vigenère cypher, 9 (Decrypt), 2 (Append to file)	The text from the input file will be decrypted with a key of *LEMON.* *T*he result will be appended to the *output* file	S6.3.2	Pass
T7	FR11	Checking that the application quits properly when 10 (Quit application) is chosen in the main menu	10 (Quit)	The program should display "*Goodbye*" and exit with an exit code of 0 (no errors)	S7	Pass
T8.1	FR2 FR4 FR6	Checking that the program can handle arguments from the command line to load specific key, input, and output files	Java -jar CodeBreaker2.jar (no arguments)	The program will load the default key, input, and output files in the root directory	S8.1	Pass
T8.2			Java -jar CodeBreaker2.jar .key /input ./output (all files specified)	The program will load the specified key, input, and output files instead of the defaults	S8.2	Pass

ID	FR (See Table 1)	Description	Inputs	Expected output	Actual output (See Section 3.1.1)	Pass/ Fail
T8.3	FR2 FR4 FR6	Checking that the program can handle arguments from the command line to load specific key, input, and output files	Java -jar CodeBreaker2.jar ./input ./output (some arguments missing)	The program will load the default key, input, and output files in the root directory	S8.3	Pass

3.1.1 Screenshots

S1.1

```
> 1
Choose a cypher
 | 1.  Caesar
 | 2.  Keyed Caesar
 | 3.  Vigenère
> 2

Cypher: Keyed Caesar
Input file: file
Choose an action
```

S1.4

```
> 1
Choose a cypher
 | 1.  Caesar
 | 2.  Keyed Caesar
 | 3.  Vigenère
> a
Choose a cypher
 | 1.  Caesar
 | 2.  Keyed Caesar
 | 3.  Vigenère
> Try again
```

S1.2

```
> 1
Choose a cypher
 | 1.  Caesar
 | 2.  Keyed Caesar
 | 3.  Vigenère
> 3

Cypher: Vigenère
Input file: file
```

S2.1

keys

7lemon

```
> 2
Integer key: 7
String key: LEMON
Stored in file as: 7lemon
```

S1.3

```
> 1
Choose a cypher
 | 1.  Caesar
 | 2.  Keyed Caesar
 | 3.  Vigenère
> 1

Cypher: Caesar
Input file: file
Choose an action
```

S2.2

```
> 3
Enter new key
(format as <integer><string>, e.g. 7lemon)
4apple
Successfully changed key

Cypher: Caesar
Input file: file
Choose an action
 | 1.  Change cypher
 | 2.  Display key
 | 3.  Change key
 | 4.  Display input file
 | 5.  Change input file
 | 6.  Display output file
 | 7.  Change output file
 | 8.  Encrypt
 | 9.  Decrypt
 | 10.  Quit application
> 2
Integer key: 4
String key: APPLE
Stored in file as: 4apple
```

keys

4apple

S3.2

input2

This is input file 2!

```
> 5
Enter path of file
input2

Cypher: Caesar
Input file: input2
Choose an action
 | 1.  Change cypher
 | 2.  Display key
 | 3.  Change key
 | 4.  Display input file
 | 5.  Change input file
 | 6.  Display output file
 | 7.  Change output file
 | 8.  Encrypt
 | 9.  Decrypt
 | 10.  Quit application
> 4
"input2" at './input2':
THISISINPUTFILE
```

S2.3

```
> 3
Enter new key
(format as <integer><string>, e.g. 7lemon)
mykey
Enter new key
(format as <integer><string>, e.g. 7lemon)
The provided key does not have a valid format, please try again
```

S3.3

```
> 5
Enter path of file
myfile
Unable to load file "myfile"
Enter path of file
```

S3.1

input

My hovercraft is full of eels

```
> 4
"input" at './input':
MYHOVERCRAFTISFULLOFEELS
```

S4.1

output

AXEEHPHKEW

```
> 6
"output" at './output':
AXEEHPHKEW
```

S4.2

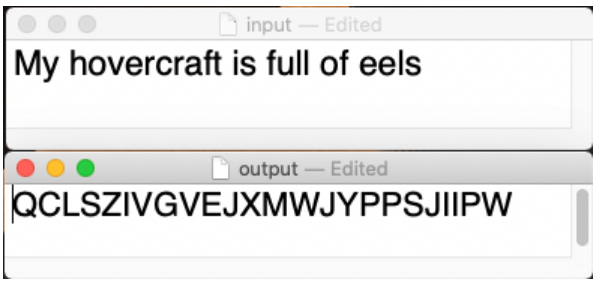
```
> 7
Enter path of file
output2

Cypher: Caesar
Input file: input
Choose an action
 | 1.  Change cypher
 | 2.  Display key
 | 3.  Change key
 | 4.  Display input file
 | 5.  Change input file
 | 6.  Display output file
 | 7.  Change output file
 | 8.  Encrypt
 | 9.  Decrypt
 | 10.  Quit application
> 6
"output2" at './output2':
```

S4.3

```
> 7
Enter path of file
myfile
Enter path of file
Unable to load file "myfile"
```

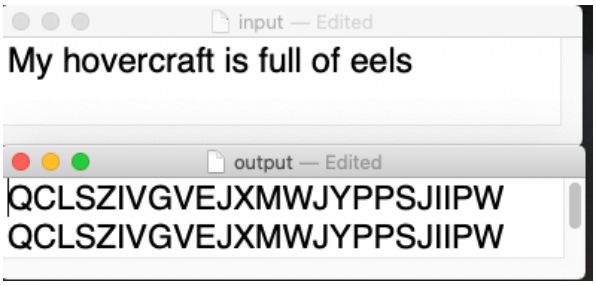
S5.1.1

```
> 8
shifted:  WXYZABCDEFGHIJKLMNOPQRSTUV
result: QCLSZIVGVEJXMWJYPPSJIIPW
Overwrite output file?
 | 1.  Yes
 | 2.  No
> 1
Successfully wrote to file output
```



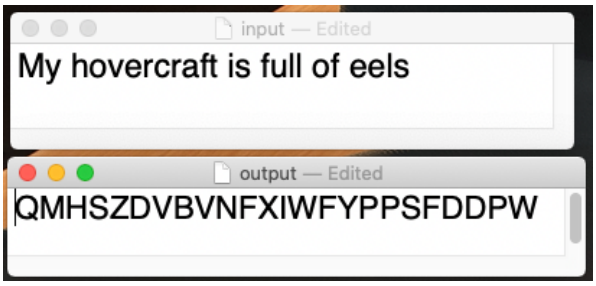
S5.1.2

```
> 8
shifted:  WXYZABCDEFGHIJKLMNOPQRSTUV
result: QCLSZIVGVEJXMWJYPPSJIIPW
Overwrite output file?
 | 1.  Yes
 | 2.  No
> 2
Successfully wrote to file output
```



S5.2.1

```
> 8

result: QMHSZDVBVNFXIWFYPPSFDDPW
Overwrite output file?
 | 1.  Yes
 | 2.  No
> 1
Successfully wrote to file output
```



S5.2.2

```
> 8

result: QMHSZDVBVNFXIWFYPPSFDDPW
Overwrite output file?
 | 1.  Yes
 | 2.  No
> 2
Successfully wrote to file output
```

input — Edited

My hovercraft is full of eels

output — Edited

QMHSZDVBVNFXIWFYPPSFDDPW
QMHSZDVBVNFXIWFYPPSFDDPW

S5.3.2

```
> 8
result: XCTCIPVOFNQXUGSFPXCSPIXG
Overwrite output file?
 | 1.  Yes
 | 2.  No
> 2
Successfully wrote to file output
```

input — Edited

My hovercraft is full of eels

output — Edited

XCTCIPVOFNQXUGSFPXCSPIXG
XCTCIPVOFNQXUGSFPXCSPIXG

S5.3.1

```
> 8
result: XCTCIPVOFNQXUGSFPXCSPIXG
Overwrite output file?
 | 1.  Yes
 | 2.  No
> 1
Successfully wrote to file output
```

input — Edited

My hovercraft is full of eels

output — Edited

XCTCIPVOFNQXUGSFPXCSPIXG

S6.1.1

```
> 9
result: MYHOVERCRAFTISFULLOFEELS
Overwrite output file?
 | 1.  Yes
 | 2.  No
> 1
Successfully wrote to file output
```

input

QCLSZIVGVEJXMWJYPPSJIIPW

output — Edited

MYHOVERCRAFTISFULLOFEELS

S6.1.2

```
> 9
result: MYHOVERCRAFTISFULLOFEELS
Overwrite output file?
 | 1.  Yes
 | 2.  No
> 2
Successfully wrote to file output
```

input

QCLSZIVGVEJXMWJYPPSJIIPW

output — Edited

MYHOVERCRAFTISFULLOFEELS
MYHOVERCRAFTISFULLOFEELS

S6.2.2

```
> 9
result: MYHOVERCRAFTISFULLOFEELS
Overwrite output file?
 | 1.  Yes
 | 2.  No
> 2
Successfully wrote to file output
```

input — Edited

QMHSZDVBVNFXIWFYPPSFDDPW

output — Edited

MYHOVERCRAFTISFULLOFEELS
MYHOVERCRAFTISFULLOFEELS

S6.2.1

```
> 9
result: MYHOVERCRAFTISFULLOFEELS
Overwrite output file?
 | 1.  Yes
 | 2.  No
> 1
Successfully wrote to file output
```

input — Edited

QMHSZDVBVNFXIWFYPPSFDDPW

output — Edited

MYHOVERCRAFTISFULLOFEELS

S6.3.1

```
> 9
result: MYHOVERCRAFTISFULLOFEELS
Overwrite output file?
 | 1.  Yes
 | 2.  No
> 1
Successfully wrote to file output
```

input — Edited

XCTCIPVOFNQXUGSFPXCSPIXG

output — Edited

MYHOVERCRAFTISFULLOFEELS

S6.3.2

```
> 9
result: MYHOVERCRAFTISFULLOFEELS
Overwrite output file?
  | 1.  Yes
  | 2.  No
> 2
Successfully wrote to file output
```

input — Edited

```
XCTCIPVOFNQXUGSFPXCSPIXG
```

output — Edited

```
MYHOVERCRAFTISFULLOFEELS
MYHOVERCRAFTISFULLOFEELS
```

S8.2

```
CodeBreaker2_jar — java -jar CodeBreaker2.jar ./keys ./input ./output — java — java -jar CodeBreaker2.jar ./keys ./input ./output — 80×...
 benbrackenbury@macbookpro  ~/Desktop/Uni Work/OOP/Main Assignment/CodeBreaker2
/out/artifacts/CodeBreaker2_jar  ls
CodeBreaker2.jar input          keys            output
 benbrackenbury@macbookpro  ~/Desktop/Uni Work/OOP/Main Assignment/CodeBreaker2
/out/artifacts/CodeBreaker2_jar  java -jar CodeBreaker2.jar ./keys ./input ./ou
tput
Cypher: Caesar
Input file: input
Choose an action
 | 1.  Change cypher
 | 2.  Display key
 | 3.  Change key
 | 4.  Display input file
 | 5.  Change input file
 | 6.  Display output file
 | 7.  Change output file
 | 8.  Encrypt
 | 9.  Decrypt
 | 10.  Quit application
> 2
Integer key: 7
String key: LEMON
Stored in file as: 7lemon

Cypher: Caesar
Input file: input
Choose an action
 | 1.  Change cypher
 | 2.  Display key
 | 3.  Change key
 | 4.  Display input file
 | 5.  Change input file
 | 6.  Display output file
 | 7.  Change output file
 | 8.  Encrypt
 | 9.  Decrypt
 | 10.  Quit application
> 6
"output" at '././output':
AXEEHPHKEW
```

S7

```
> 10

Goodbye

Process finished with exit code 0
```

S8.3

```
CodeBreaker2_jar — java -jar CodeBreaker2.jar ./input ./output — java — java -jar CodeBreaker2.jar ./input ./output — 80×40
 benbrackenbury@macbookpro  ~/Desktop/Uni Work/OOP/Main Assignment/CodeBreaker2
/out/artifacts/CodeBreaker2_jar  ls
CodeBreaker2.jar input          keys            output
 benbrackenbury@macbookpro  ~/Desktop/Uni Work/OOP/Main Assignment/CodeBreaker2
/out/artifacts/CodeBreaker2_jar  java -jar CodeBreaker2.jar ./input ./output
Cypher: Caesar
Input file: input
Choose an action
 | 1.  Change cypher
 | 2.  Display key
 | 3.  Change key
 | 4.  Display input file
 | 5.  Change input file
 | 6.  Display output file
 | 7.  Change output file
 | 8.  Encrypt
 | 9.  Decrypt
 | 10.  Quit application
> 2
Integer key: 7
String key: LEMON
Stored in file as: 7lemon

Cypher: Caesar
Input file: input
Choose an action
 | 1.  Change cypher
 | 2.  Display key
 | 3.  Change key
 | 4.  Display input file
 | 5.  Change input file
 | 6.  Display output file
 | 7.  Change output file
 | 8.  Encrypt
 | 9.  Decrypt
 | 10.  Quit application
> 6
"output" at './output':
AXEEHPHKEW
```

S8.1

```
CodeBreaker2_jar — java -jar CodeBreaker2.jar — java — java -jar CodeBreaker2.jar — 80×40
 benbrackenbury@macbookpro  ~/Desktop/Uni Work/OOP/Main Assignment/CodeBreaker2
/out/artifacts/CodeBreaker2_jar  ls                                    r
CodeBreaker2.jar input          keys            output
 benbrackenbury@macbookpro  ~/Desktop/Uni Work/OOP/Main Assignment/CodeBreaker2
/out/artifacts/CodeBreaker2_jar  java -jar CodeBreaker2.jar
Cypher: Caesar
Input file: input
Choose an action
 | 1.  Change cypher
 | 2.  Display key
 | 3.  Change key
 | 4.  Display input file
 | 5.  Change input file
 | 6.  Display output file
 | 7.  Change output file
 | 8.  Encrypt
 | 9.  Decrypt
 | 10.  Quit application
> 2
Integer key: 7
String key: LEMON
Stored in file as: 7lemon

Cypher: Caesar
Input file: input
Choose an action
 | 1.  Change cypher
 | 2.  Display key
 | 3.  Change key
 | 4.  Display input file
 | 5.  Change input file
 | 6.  Display output file
 | 7.  Change output file
 | 8.  Encrypt
 | 9.  Decrypt
 | 10.  Quit application
> 6
"output" at './output':
AXEEHPHKEW
```

3.1.2 Test Discussion

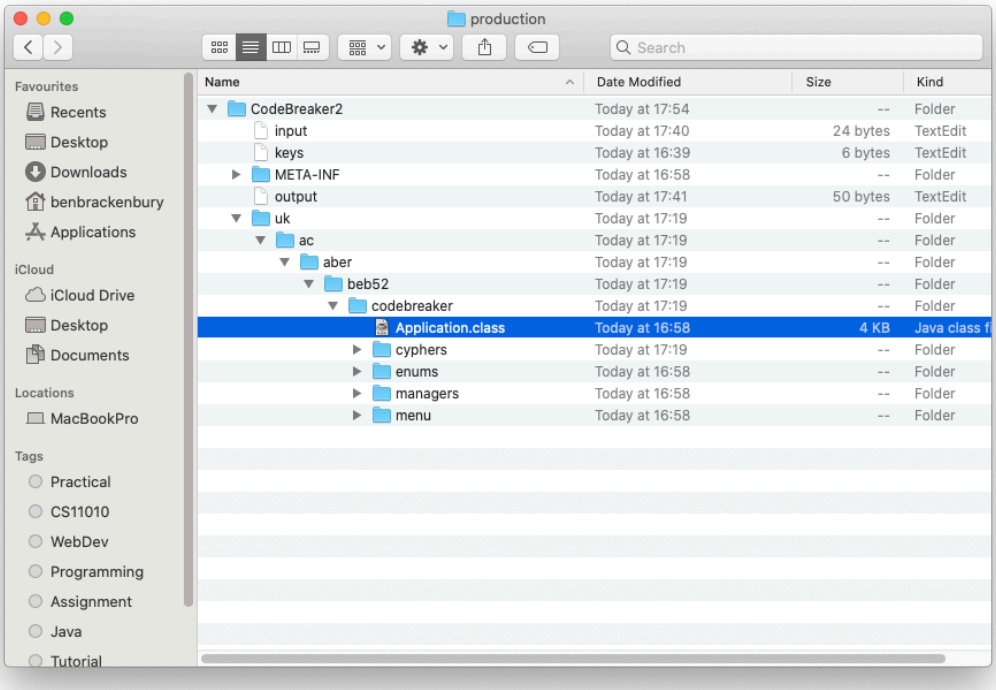
Every test in my test plan passed, which means that my application is robust and has no major errors.

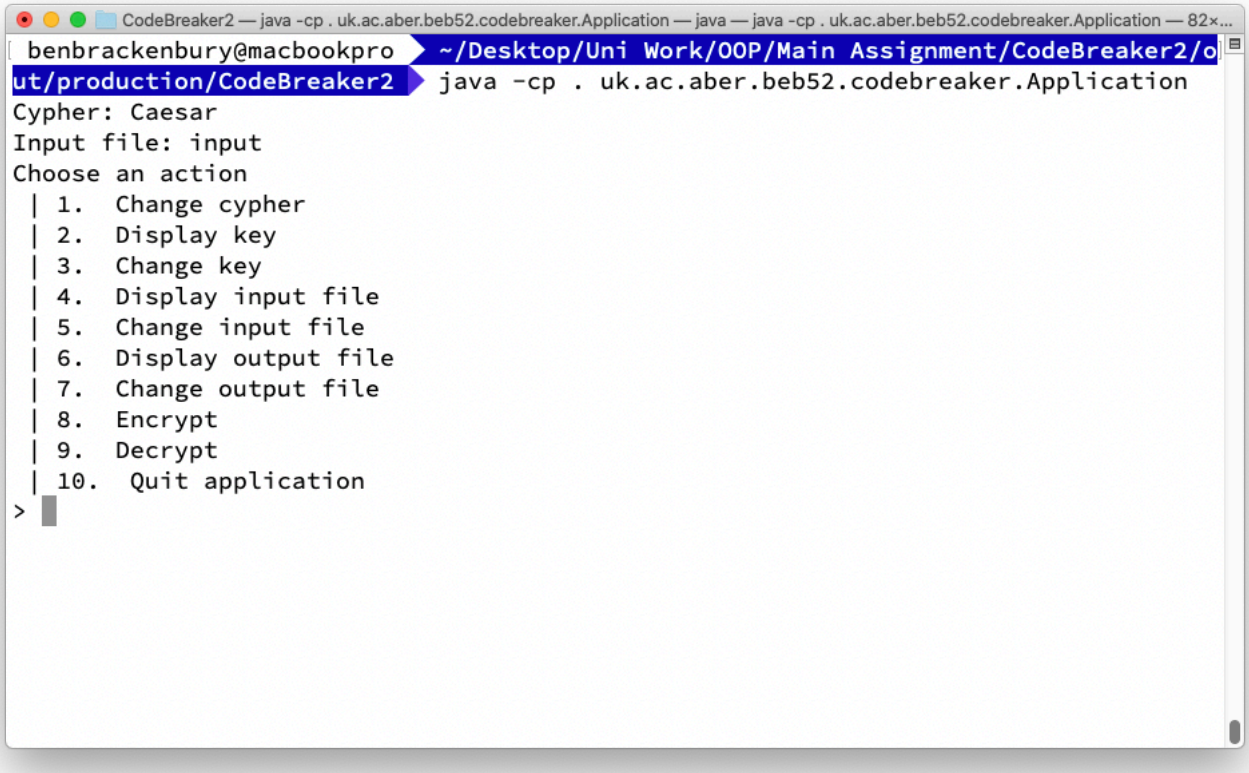
Since the majority of the input to the program is via a menu system, the scope of input is limited. As shown in **Screenshot 1.4**, if an invalid option is input into a menu (whether the key is out of range or a non-number is provided), an error message tells the user that their input is invalid and re-prompts for another try. The same thing happens when the user inputs the path of a non-existent file, as shown in **Screenshot 3.3**.

Similarly, if the users inputs a new key to use, their input is validated, checking that the key they have provided is in the correct format. This is shown in **Screenshot 2.3**.

3.2 Command Line

Screenshots of running application from command line





4 Evaluation

4.1 Solving the assignment

I started by reading through the assignment brief. Once I fully understood the task and its requirements, I began to abstract the problem, thinking about use classes and relationships.

The first code I wrote for the assignment was the *Application* class and *main* method, since it is the entry point of the program. I then focussed on the menu system, then the manager classes, then classes related to the cyphers.

While I didn't encounter any major difficulties, I found the encryption and decryption algorithms for the Vigenère cypher the most challenging, as they were the most complex.

4.2 Flair

I generated JavaDoc HTML pages from the code's comments; this can be viewed in the project's *docs* folder.

I decided the add some additional functionality to the application, where users can specify the paths of files to use for keys, input, and output as optional arguments in the command line. This gives users more choice and they do not have to understand how the default file paths are set up to use the application. This functionality is shown in **Screenshot 8.2**.

4.3 Challenges

As mentioned above in **Section 4.1**, I ran into no major difficulties, however, the Vigenère encryption and decryption algorithms were the most complex compared to my more simple algorithms for the two Caesar cyphers.

4.4 What remains to be done

As mentioned in **Section 1**, I feel that I have completed this assignment and have met all of the functional requirements. However, if I had more time, I would likely have added something more substantial like a graphical user interface for additional flair. A GUI would improve the ease of use of my application, as they are more user friendly and easy to understand than text based interfaces.

4.5 What was learnt

I feel like completing this assignment made me appreciate inheritance and polymorphism more, as it can dramatically reduce the amount of code required, and allows for greater extensibility in the future.

4.6 Estimated mark

I fell like I deserve around 70% for this assignment, since I think that my documentation is of a high quality and my code follows OOP principles quite well, using inheritance and polymorphism effectively. I feel that my identifier names are understandable and my code is robust. My testing plan covers every function of the program and shows clear evidence of testing.